The semantic function tool is used in the tree's experts, nodes, and answer blocks. At each step, the function determines if any text is being input. If there is no text, the block returns with no execution. Passing no text to a block means that the previous block failed evaluation. By evaluating before each step, ToT short-circuits the execution of plans it deems as not being valid.

This may be a complex pattern to grasp at first, so go ahead and run the flow in VS Code. Listing 10.20 shows just the answer node output of a run; these results may vary from what you see but should be similar. Nodes that return no text either failed evaluation or their parents did.

Listing 10.20 Output from `tree-of-thought-evaluation` flow

```
{
    "answer_1_1": "",     #1
    "answer_1_2": "",
    "answer_1_3": "",
    "answer_2_1": "Alex spends a total of 29 days in the past before he
sees the end of the battle.",
    "answer_2_2": "",     #2
    "answer_2_3": "Alex spends a total of 29 days in the past before he
sees the end of the battle.",
    "answer_3_1": "",     #3
    "answer_3_2": "Alex spends a total of 29 days in the past before he
sees the end of the battle.",
    "answer_3_3": "Alex spends a total of 9 days in the past before he
sees the end of the battle.",
```

#1 Represents that the first node plans weren't valid and not executed
#2 The plan for node 2 and answer 2 failed evaluation and wasn't run.
#3 The plan for this node failed to evaluate and wasn't run.

The output in listing 10.20 shows how only a select set of nodes was evaluated. In most cases, the evaluated nodes returned an answer that could be valid. Where no output was produced, it means that the node itself or its parent wasn't valid. When sibling nodes all return empty, the parent node fails to evaluate.

As we can see, ToT is valid for complex problems but perhaps not very practical. The execution of this flow can take up to 27 calls to an LLM to generate an output. In practice, it may only do half that many calls, but that's still a dozen or more calls to answer a single problem.